

KEYS

1. Windows+ X = All system details
2. Alt + tab = Another window,
(If use shift,it can move forward and backward))
3. Windows R = run dialogue (command)
4. Windows+ H = voice to text in (microsoft word)
5. Windows+shift+s = snipping tool (edit screenshot while we take screenshot)
6. Windows +. = emoji (can use in word,notion,notepad)
7. ctrl + shift + esc = Task Manager
8. Alt + ctrl + dlt = when use in the system hanging time
9. Alt + f4 = minimize , alt + fn + f4 = all close
10. If we play screenrecord,its recording time less than 30 seconds,can make GIF
11. windows alt R = Instant record
12. Windows + G = Capture details+ record screen option
13. Cntrl shift arrow = select sentences, shift arrow also work
14. Cntrl down arrow = select nearest sentence also
15. Docs.new (search in chrome) = google new document
16. In documents:
 - 1.share criterias : Can restrict by view or not viewing for others
 - 2.clock icon,we can check changes(it can view)
 - 3.multiple person can access,give request - it will be get through him in mail
 - 4.@today - today's date
 - 5.@yesterday - yesterday's date
 6. file - new - template
 - 7.insert -symbol- eg: rupee symbol
 - 8 ctrl enter- next page
 - 9 file-page- edit page as portrait,page less, make titles-heading (alt ctrl 1),make sub headings
 - 10 if we need to paste a paragraph or screenshot,in the same format of doc- cntrl shift v
 11. ctrl k - link copy - site or pic anything
 - 12.ctrl forward slash- shortcuts
 - 13.ctrl alt f- can add foot note
 - 14.ctrl F - FIND any particular word
 - 15.ctrl H- FIND and REPLACE any word
 - 16 Tab- can move cursor instead of mouse (used in any website)
 - 17.Windows- accessibility - keyboard virtual
 18. word file can convert into docs by upload in on drive
17. **Notebooklm** (app) - if a pdf or file is large it can convert into short,specific details can ask
18. If need any pdf file use or search - filetype:pdf
19. Ctrl shift T = Redo close tab if we unexpectedly close

Data Science - Dr Ignez Semmelweiz

- **Data Science used in** Travel, Healthcare, Sales, Credit & Media, Marketing, Social Media, Automation
- **Automation-** self driving cars, Pilotless aircrafts,drones
- **Credit& Insurance** - Claims prediction, Fraud & Risk Detection
- **Marketing-** Upselling,cross selling,churn- churn meaning in business (leaving or stopping subscription)
- Data Science includes **AI, ML, Deep Learning, Maths, Statistics, Visualization, EDA**

Why ML for future - 460+ exabytes data generate every day based on 2025

- **1 byte= 8 bit**
- **1000 such letters form 1 kilobyte**

MACHINE LEARNING

1. Supervised learning - label image -train
- classification and regression

classification- predicting a category or class.

Eg: It is tiger or cat finding, email spam detection

Regression - predicting continuous numerical values

Eg: House price-400000, salary-20000, 45 degree temperature.

2. Unsupervised learning- no labels,machine will figure out some factors
-clustering and association includes here

Clustering-grouping similar data

Association- finding relationships or rules b/w items

😊 There is a site- "Teachable Machine" to train model

⇒ Machine Learning Applications-

- Virtual Assistant
- eCommerce
- Financial Services
- social media
- Health care

PYTHON

Python is a High level, object oriented, interpreter,Dynamic semantic, open source language

1.Numpy - is a python library for multi dimesional array processing- raw data handle(calculation,mathematical operation)

2.Pandas -Built on top of numpy,which provides multi dimesional array processing- data analysing,cleaning,data manipulation

3.scikit learn - machine library in python- (clustering,regression process)

AI

- Turing Test

♥ Process of DATA SCIENCE

- Data Mining- collecting relevant data
- Data Cleaning- handling missing data

- **Data Exploration- (EDA) Form hypothesis about defined problems (means observe data and making possible assumptions or predictions)**
- **Feature Engineering - eg: weekend, lot of people purchase a particular thing(finding through feature engineering (Changing or creating data columns in a better way)**
- **Predictive Modelling -training a model using old data to predict future results)**
- **Data Visualization**

To check python install?

Select terminal -> `python --version` OR `py -V`

If we get the version ,python is installed

Python style -snake case

JavaScript - Camel case

Virtual environment -venv(in built in python)

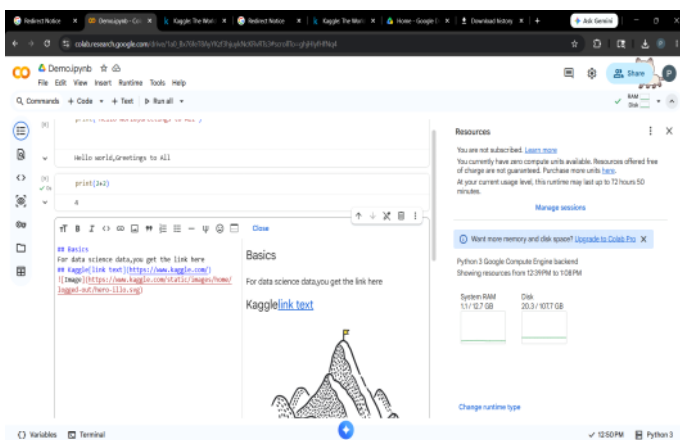
For Code demo use "[google colab.com](https://colab.research.google.com)" (chrome search)

Insert link is okay there

If we link any image copy address,image can see at right side by adding ! In left of image le,

Put `![Image]` then copy image address or link,we can see image at right side

Can see this in the screenshot 😊



In Google colab-can get any extra data through drive,

It is get by `files-> mount drive-` can get data

1. In colab,if we need command in code,Add a `#` symbol or `/` before the text
2. If I need output in multiple variables use bracket with comma `(4,5,6)` or `("red","yellow","blue")`
- 3.Reuse ,store->`variables`-it is a reserved memory location

Eg: `age = 12`

`print(age)`

Output= 12

4. Variables must be start with **letter or underscore, 0-9**

5. Variables can't be a keyword like **print**

6. **Case sensitive**

Modulus %- find reminder

Exponent *- find power

Floor division //-divide numbers, also remove the decimal part.eg: if 3.444,

OPERATORS

== Equal to

!= not equal

Relational operators

-

In set, we can't add duplicate elements,slicing,is only mutable,it is ordered

List functions

Reverse function

Iteration method- to avoid repeated print

-definite loop-for loop- one condition,1 increment,1 decrement

-indefinite loop-while loop:initialization, condition with relational operations,increment or decrement

More than conditions combine- using logical operations

User Defined Function in python

-built in fn

-

Control statements,mainly 3 statements in python

- Break,

- Continue,

- Pass-

Function call eg:mul()- inside the () is called the arguments

Actual parameter

Formal parameter

OOPS

Map fn-

Object and class

Inside the class, written fn- called->methods, variables called attributes

Inheritance-multiple objects create-parents class->inherit to child class

Multilevel inheritance

-class access-key called objects

Version control Git & Github

02 June 2026 09:26

Version control repository- Local(PC/Laptop)

Remote control repository- Remote(Server)

1. Git status-git status is a Git command that shows the current state of your repository.
2. git init stands for **Git Initialize**.
It creates a new Git repository in your current folder.
3. git add
git commit -m "added file"
git add-second line add
git commit -m "added line1"
- 3 git log
- 4.git checkout
- 5.git switch
- 6 git branch
- 6.git checkout master
- 7.git branch
- 8.git branch dev
- 9.git checkout dev
- 10.git clone URL- code from github repository
11. Git push
12. Fork
13. Pull request

Mean-Average

Median- Middle value

Mean, median, mode together called central tendency

Mode- Most repeated value

-> Standard Deviation :Standard Deviation (SD) measures how spread out the data is from the mean (average).

Eg:Class A

Marks:

48, 49, 50, 51, 52

Mean = 50

Distance from mean:

- 48 → 2 below mean
- 49 → 1 below mean
- 50 → exactly mean
- 51 → 1 above mean
- 52 → 2 above mean

All values are close to the mean.

➡ **Standard deviation is small.**

Class B

Marks:

20, 35, 50, 65, 80

Mean = 50

Distance from mean:

- 20 → 30 below mean
- 35 → 15 below mean
- 50 → exactly mean
- 65 → 15 above mean
- 80 → 30 above mean

Values are much farther from the mean.

➡ **Standard deviation is large.**

Eg: **2, 4, 6, 8, 10**

Step 1: Mean

$$\text{Mean} = \frac{2 + 4 + 6 + 8 + 10}{5} = 6$$

Step 2: Mean-il ninn difference

Value	Difference
2	-4
4	-2
6	0
8	2
10	4

Step 3: Difference square cheyyuka

Difference	Square
-4	16
-2	4
0	0
2	4
4	16

Step 4: Squares add cheyyuka

$$16 + 4 + 0 + 4 + 16 = 40$$

Step 5: Number of values kondu divide cheyyuka

$$\text{Variance} = \frac{40}{5} = 8$$

Answer

Variance = 8

Variance vs SD

- **Variance** = Average of squared deviations from mean.
- **Standard Deviation** = Square root of variance.

$$SD = \sqrt{\text{Variance}}$$

Inside:

- Variance = 8
- SD = $\sqrt{8} = 2.83$

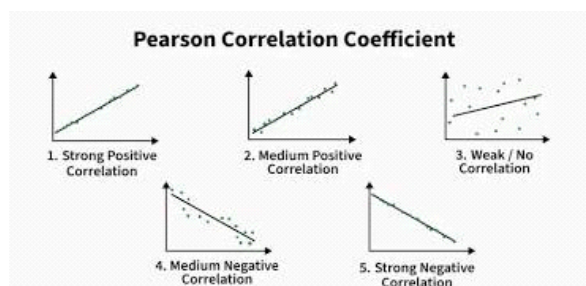
Statistics is mainly two types:

-**Descriptive** - Measures such as mean, median, mode, range, variance, standard deviation and correlation

-**Inferential**- mostly used in data science

-**Pearson's correlation coefficient** (often denoted by r) is a statistical measure that quantifies the strength and direction of the linear relationship between two continuous variables

$$r = \frac{n(\sum xy) - (\sum x)(\sum y)}{\sqrt{[n\sum x^2 - (\sum x)^2][n\sum y^2 - (\sum y)^2]}}$$



Placebo-

A placebo is any treatment that has no active properties, such as a sugar pill.
It is a dummy treatment

From <<https://www.betterhealth.vic.gov.au/health/conditionsandtreatments/placebo-effect>>

A **placebo** is a treatment that has no active therapeutic ingredient but is given in a way that makes a person believe they are receiving a real treatment.

Hypothesis- Assumption that is neither proven nor disproven

- Hypothesis is made at very beginning and the goal is to reject or not reject
- Hypothesis is in two types:
 1. H_0 = Null hypothesis
 2. H_1, H_a = Alternative hypothesis

The Null Hypothesis is the starting assumption in hypothesis testing. It states that there is no effect, no difference or no relationship between variables.

- Assumes the current situation remains unchanged
- States no change or no impact
- Directly tested using sample data
- Written using equality symbols ($=$, $\geq$, $\leq$)

From <<https://www.geeksforgeeks.org/data-science/null-vs-alternate-hypothesis/>>

The Alternative Hypothesis is the statement that challenges the null hypothesis. It suggests that an effect, a difference or a relationship does exist.

- Represents the research or study claim
- Indicates that a change or effect is present
- Accepted only when the null hypothesis (H_0) is rejected
- Written using inequality symbols ($\neq$, $>$, $<$)

From <<https://www.geeksforgeeks.org/data-science/null-vs-alternate-hypothesis/>>

Hypothesis Testing: Either we **reject the null hypothesis** or **fail to reject the null hypothesis**

Reject the null hypothesis means- accepting alternative hypothesis

Fail to reject null hypothesis means- accepting the null hypothesis

Eg: Dr would not prescribe diabetic medicine until your blood report says you are diabetic

->Significance level

In statistics, the **significance level** (denoted by α , alpha) is the probability of rejecting the null hypothesis when it is actually true (a **Type I error**).

Common significance levels are:

- $\alpha = 0.05$ (5%) — most commonly used
- $\alpha = 0.01$ (1%) — stricter evidence required
- $\alpha = 0.10$ (10%) — less strict, sometimes used in exploratory studies

The decision rule is:

- If **p-value** $\leq \alpha$, reject the null hypothesis.
- If **p-value** $> \alpha$, fail to reject the null hypothesis.

The significance level is represented by:

α

Example:

Suppose $\alpha = 0.05$ and a hypothesis test gives a p-value of 0.03.

Since **0.03 < 0.05**, you reject the null hypothesis and conclude that the result is **statistically significant** at the 5% level.

In simple terms, a significance level of 0.05 means you are willing to accept a **5% chance of falsely concluding that an effect exists when it actually does not**.

Significance level (α) is the maximum probability of rejecting a true null hypothesis (Type I error).

- **p-value** = Actual probability that the result occurred by random chance.
- **Significance level (α)** = Cutoff value we choose (usually 0.05).

Decision:

- $p\text{-value} \leq \alpha \rightarrow$ Reject H_0
- $p\text{-value} > \alpha \rightarrow$ Do not reject H_0

So, "**possibility of randomness**" is closer to the **p-value**, while the **significance level** is the **allowed limit for that possibility**

Example of Significance Level

A company claims that the average battery life of its product is **10 hours**.

- **Null Hypothesis (H_0):** Average battery life = 10 hours
- **Alternative Hypothesis (H_1):** Average battery life $\neq$ 10 hours

You choose a **significance level** of:

$\alpha = 0.05$

This means:

"I am willing to accept a 5% risk of wrongly rejecting the null hypothesis."

After collecting data and performing a test, you get:

- **p-value = 0.03**

Since:

$0.03 < 0.05$

you **reject the null hypothesis**.

Interpretation

Because the p-value is smaller than the significance level, the observed result is considered too unlikely to be explained by random chance alone.

Easy way to remember

Imagine a teacher says:

"If the chance that this happened randomly is less than 5%, I'll consider it significant."

Here:

- **5%** = Significance level
- **Actual chance from the data** = p-value

If the actual chance (p-value) is below 5%, reject H_0 . If it's above 5%, don't reject H_0 .

P-value Formula

The P-value formula is short for probability value. P-value defines the probability of getting a result that is either the same or more extreme than the other actual observations. The P-value represents the probability of occurrence of the given event. The P-value formula is used as an alternative to the rejection point to provide the least significance for which the null hypothesis would be rejected. The smaller the P-value, the stronger is the evidence in favor of the alternative hypothesis given observed frequency and expected frequency.

What is P-value Formula?

P-value is an important statistical measure, that helps to determine whether the hypothesis is correct or not. **P-value always only lies between 0 and 1**. The level of significance(α) is a predefined threshold that should be set by the researcher. It is generally fixed as 0.05

From <<https://www.cuemath.com/p-value-formula/>>

Confidence Level = The percentage that indicates how confident we are that the sample result represents the true population parameter.

For most data analytics problems:

- 95% confidence level = standard choice
- 99% confidence level = more strict
- 90% confidence level = less strict

Eg: Suppose you want to know the average height of **10,000 students**.

You can't measure everyone, so you take a sample of 100 students.

After calculation, you estimate:

Average height $\approx$ 165 cm

Now the question is:

"How sure am I that this estimate is close to the true average height of all 10,000 students?"

If you say **95% confidence level**, it means:

"I am 95% confident that my estimation method is correct."

The remaining **5%** is the chance that your sample gave a misleading result.

So:

- **Confidence Level = 95%** $\rightarrow$ Confidence in the result.
- **Significance Level = 5%** $\rightarrow$ Risk of being wrong.

Zscore

A **Z-score (Standard Score)** tells you **how many standard deviations a data point is away from the mean**.

$$z = \frac{x - \mu}{\sigma}$$

- **z** = Z-score
- **x** = data value
- **μ (mu)** = mean
- **σ (sigma)** = standard deviation

Z-Score Formula (Not a Hypothesis Test)

Use when:

- Finding how far a single value is from the mean

P- Value formula

STANDARD NORMAL PROBABILITIES

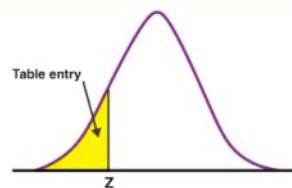


Table entry for **z** is the area under the standard normal curve to the left of **z**.

Z	.00	.01	.02	.03	.04	.05	.06	.07	.08	.09
-3.4	.0003	.0003	.0003	.0003	.0003	.0003	.0003	.0003	.0003	.0002
-3.3	.0005	.0005	.0005	.0004	.0004	.0004	.0004	.0004	.0004	.0003
-3.2	.0007	.0007	.0006	.0006	.0006	.0006	.0006	.0005	.0005	.0005
-3.1	.0010	.0009	.0009	.0009	.0008	.0008	.0008	.0008	.0007	.0007
-3.0	.0013	.0013	.0013	.0012	.0012	.0011	.0011	.0011	.0010	.0010
-2.9	.0019	.0018	.0018	.0017	.0016	.0016	.0015	.0015	.0014	.0014
-2.8	.0026	.0025	.0024	.0023	.0023	.0022	.0021	.0021	.0020	.0019
-2.7	.0035	.0034	.0033	.0032	.0031	.0030	.0029	.0028	.0027	.0026
-2.6	.0047	.0045	.0044	.0043	.0041	.0040	.0039	.0038	.0037	.0036
-2.5	.0062	.0060	.0059	.0057	.0055	.0054	.0052	.0051	.0049	.0048
-2.4	.0082	.0080	.0078	.0075	.0073	.0071	.0069	.0068	.0066	.0064
-2.3	.0107	.0104	.0102	.0099	.0096	.0094	.0091	.0089	.0087	.0084
-2.2	.0139	.0136	.0132	.0129	.0125	.0122	.0119	.0116	.0113	.0110
-2.1	.0179	.0174	.0170	.0166	.0162	.0158	.0154	.0150	.0146	.0143

-2.0	.0228	.0222	.0217	.0212	.0207	.0202	.0197	.0192	.0188	.0183
-1.9	.0287	.0281	.0274	.0268	.0262	.0256	.0250	.0244	.0239	.0233
-1.8	.0359	.0351	.0344	.0336	.0329	.0322	.0314	.0307	.0301	.0294
-1.7	.0446	.0436	.0427	.0418	.0409	.0401	.0392	.0384	.0375	.0367
-1.6	.0548	.0537	.0526	.0516	.0505	.0495	.0485	.0475	.0465	.0455
-1.5	.0668	.0655	.0643	.0630	.0618	.0606	.0594	.0582	.0571	.0559
-1.4	.0808	.0793	.0778	.0764	.0749	.0735	.0721	.0708	.0694	.0681
-1.3	.0968	.0951	.0934	.0918	.0901	.0885	.0869	.0853	.0838	.0823
-1.2	.1151	.1131	.1112	.1093	.1075	.1056	.1038	.1020	.1003	.0985
-1.1	.1357	.1335	.1314	.1292	.1271	.1251	.1230	.1210	.1190	.1170
-1.0	.1587	.1562	.1539	.1515	.1492	.1469	.1446	.1423	.1401	.1379
-0.9	.1841	.1814	.1788	.1762	.1736	.1711	.1685	.1660	.1635	.1611
-0.8	.2119	.2090	.2061	.2033	.2005	.1977	.1949	.1922	.1894	.1867
-0.7	.2420	.2389	.2358	.2327	.2296	.2266	.2236	.2206	.2177	.2148
-0.6	.2743	.2709	.2676	.2643	.2611	.2578	.2546	.2514	.2483	.2451
-0.5	.3085	.3050	.3015	.2981	.2946	.2912	.2877	.2843	.2810	.2776
-0.4	.3446	.3409	.3372	.3336	.3300	.3264	.3228	.3192	.3156	.3121
-0.3	.3821	.3783	.3745	.3707	.3669	.3632	.3594	.3557	.3520	.3483
-0.2	.4207	.4168	.4129	.4090	.4052	.4013	.3974	.3936	.3897	.3859
-0.1	.4602	.4562	.4522	.4483	.4443	.4404	.4364	.4325	.4286	.4247
-0.0	.5000	.4960	.4920	.4880	.4840	.4801	.4761	.4721	.4681	.4641

Standard Normal Probabilities

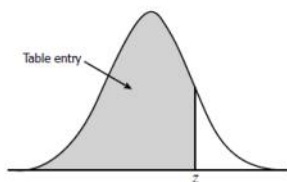


Table entry for z is the area under the standard normal curve to the left of z .

z	.00	.01	.02	.03	.04	.05	.06	.07	.08	.09
0.0	.5000	.5040	.5080	.5120	.5160	.5199	.5239	.5279	.5319	.5359
0.1	.5398	.5438	.5478	.5517	.5557	.5596	.5636	.5675	.5714	.5753
0.2	.5793	.5832	.5871	.5910	.5948	.5987	.6026	.6064	.6103	.6141
0.3	.6179	.6217	.6255	.6293	.6331	.6368	.6406	.6443	.6480	.6517
0.4	.6554	.6591	.6628	.6664	.6700	.6736	.6772	.6808	.6844	.6879
0.5	.6915	.6950	.6985	.7019	.7054	.7088	.7123	.7157	.7190	.7224
0.6	.7257	.7291	.7324	.7357	.7389	.7422	.7454	.7486	.7517	.7549
0.7	.7580	.7611	.7642	.7673	.7704	.7734	.7764	.7794	.7823	.7852
0.8	.7881	.7910	.7939	.7967	.7995	.8023	.8051	.8078	.8106	.8133
0.9	.8159	.8186	.8212	.8238	.8264	.8289	.8315	.8340	.8365	.8389
1.0	.8413	.8438	.8461	.8485	.8508	.8531	.8554	.8577	.8599	.8621
1.1	.8643	.8665	.8686	.8708	.8729	.8749	.8770	.8790	.8810	.8830
1.2	.8849	.8869	.8888	.8907	.8925	.8944	.8962	.8980	.8997	.9015
1.3	.9032	.9049	.9066	.9082	.9099	.9115	.9131	.9147	.9162	.9177
1.4	.9192	.9207	.9222	.9236	.9251	.9265	.9279	.9292	.9306	.9319
1.5	.9332	.9345	.9357	.9370	.9382	.9394	.9406	.9418	.9429	.9441
1.6	.9452	.9463	.9474	.9484	.9495	.9505	.9515	.9525	.9535	.9545
1.7	.9554	.9564	.9573	.9582	.9591	.9599	.9608	.9616	.9625	.9633
1.8	.9641	.9649	.9656	.9664	.9671	.9678	.9686	.9693	.9699	.9706
1.9	.9713	.9719	.9726	.9732	.9738	.9744	.9750	.9756	.9761	.9767
2.0	.9772	.9778	.9783	.9788	.9793	.9798	.9803	.9808	.9812	.9817
2.1	.9821	.9826	.9830	.9834	.9838	.9842	.9846	.9850	.9854	.9857
2.2	.9861	.9864	.9868	.9871	.9875	.9878	.9881	.9884	.9887	.9890
2.3	.9893	.9896	.9898	.9901	.9904	.9906	.9909	.9911	.9913	.9916
2.4	.9918	.9920	.9922	.9925	.9927	.9929	.9931	.9932	.9934	.9936
2.5	.9938	.9940	.9941	.9943	.9945	.9946	.9948	.9949	.9951	.9952
2.6	.9953	.9955	.9956	.9957	.9959	.9960	.9961	.9962	.9963	.9964
2.7	.9965	.9966	.9967	.9968	.9969	.9970	.9971	.9972	.9973	.9974
2.8	.9974	.9975	.9976	.9977	.9977	.9978	.9979	.9979	.9980	.9981
2.9	.9981	.9982	.9982	.9983	.9984	.9984	.9985	.9985	.9986	.9986
3.0	.9987	.9987	.9987	.9988	.9988	.9989	.9989	.9989	.9990	.9990
3.1	.9990	.9991	.9991	.9991	.9992	.9992	.9992	.9992	.9993	.9993
3.2	.9993	.9993	.9994	.9994	.9994	.9994	.9995	.9995	.9995	.9995
3.3	.9995	.9995	.9995	.9996	.9996	.9996	.9996	.9996	.9996	.9997
3.4	.9997	.9997	.9997	.9997	.9997	.9997	.9997	.9997	.9997	.9998

1. $-2.97 - .0015$

Hypothesis testing

- Different types of testing:

1.Z Test

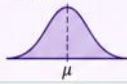
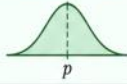
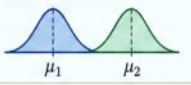
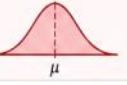
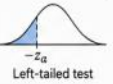
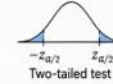
One-Sample Z-Test for Mean

Use when:

- Population SD (σ) is known

- Testing an average

Z-TEST FORMULAS

TEST TYPE	FORMULA	WHEN TO USE	NOTES
1. ONE-SAMPLE Z-TEST FOR MEAN 	$z = \frac{\bar{x} - \mu_0}{\sigma / \sqrt{n}}$	- Population standard deviation (σ) is known - Testing a population mean (average)	$\bar{x}$ = sample mean μ_0 = population mean (claimed) σ = population standard deviation n = sample size
2. ONE-SAMPLE Z-TEST FOR PROPORTION 	$z = \frac{\hat{p} - p_0}{\sqrt{\frac{p_0(1-p_0)}{n}}}$	- Testing a population proportion (percentage) - Large sample ($np_0 \geq 5$ and $n(1-p_0) \geq 5$)	$\hat{p}$ = sample proportion (x/n) p_0 = population proportion (claimed) n = sample size x = number of successes
3. TWO-SAMPLE Z-TEST FOR MEANS 	$z = \frac{\bar{x}_1 - \bar{x}_2}{\sqrt{\frac{\sigma_1^2}{n_1} + \frac{\sigma_2^2}{n_2}}}$	- Comparing means of two independent populations - Population SDs (σ_1, σ_2) are known	$\bar{x}_1, \bar{x}_2$ = sample means σ_1, σ_2 = population standard deviations n_1, n_2 = sample sizes
4. TWO-PROPORTION Z-TEST 	$z = \frac{\hat{p}_1 - \hat{p}_2}{\sqrt{\hat{p}(1-\hat{p})\left(\frac{1}{n_1} + \frac{1}{n_2}\right)}}$ where $\hat{p} = \frac{x_1 + x_2}{n_1 + n_2}$	- Comparing proportions of two independent populations - Large samples	$\hat{p}_1, \hat{p}_2$ = sample proportions ($x_1/n_1, x_2/n_2$) $\hat{p}$ = pooled proportion n_1, n_2 = sample sizes x_1, x_2 = number of successes
5. Z-SCORE FORMULA (NOT A HYPOTHESIS TEST) 	$z = \frac{x - \mu}{\sigma}$	- Find how far a single value (x) is from the mean - Standardization - Finding probabilities in normal distribution	x = individual value μ = population mean σ = population standard deviation
TYPES OF Z-TESTS (BY HYPOTHESIS)   		COMMON SIGNIFICANCE LEVELS (α) 0.10 $\rightarrow z = 1.645$ 0.05 $\rightarrow z = 1.960$ 0.01 $\rightarrow z = 2.576$ 0.001 $\rightarrow z = 3.291$	

Population Mean (μ)

Question-il **already given claim/reference value** aanu.

Keywords:

- "The company claims..."
- "The average is..."
- "The population mean is..."

Example:

"The average salary is ₹30,000."

→ $\mu = 30000$ (Population Mean)

2. Sample Mean ($\bar{x}$)

Question-il **sample eduthu calculate cheytha average** aanu.

Keywords:

- "A sample of 25 employees..."
- "The sample mean was..."
- "The average of the sampled data..."

Example:

"A sample of 25 employees had an average salary of ₹32,000."

→ $\bar{x} = 32000$ (Sample Mean)

Easy Memory Trick

μ (**mu**) = **Mentioned/Market claim**

- Already known value
- Population value

$\bar{x}$ (**x bar**) = **eXperiment value**

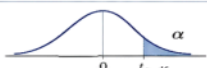
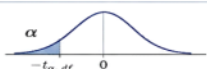
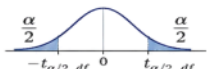
- Sample-il ninn kittiya value
- Nammal collect cheytha data

T Test

$$t = \frac{\bar{x} - \mu}{\frac{s}{\sqrt{n}}}$$

t-TEST: STEPS (FOR ONE-SAMPLE t-TEST FOR MEAN)

1 State the Hypotheses	$H_0: \mu = \mu_0$ $H_1: \text{depends on the claim}$ $\begin{cases} \mu > \mu_0 & \text{(Right-tailed test)} \\ \mu < \mu_0 & \text{(Left-tailed test)} \\ \mu \neq \mu_0 & \text{(Two-tailed test)} \end{cases}$
2 Choose the Significance Level (α)	Select α (commonly 0.05 or 0.01).
3 Check the Conditions	- Random sample - Population approximately normal OR sample size $n \geq 30$ (by Central Limit Theorem) - Independence of observations
4 Compute the Test Statistic	$t = \frac{\bar{x} - \mu_0}{s/\sqrt{n}}$ where $\bar{x}$ = sample mean μ_0 = population mean (claimed) s = sample standard deviation n = sample size
5 Determine the Degrees of Freedom (df)	$df = n - 1$
6 Find the P-value (Using t-table or software)	- Right-tailed test: $p = P(T > t_{calc})$ - Left-tailed test: $p = P(T < t_{calc})$ - Two-tailed test: $p = 2P(T > t_{calc})$ (If using t-table and exact p-value is not available, find the range: p lies between two tabulated values.)
7 Make the Decision	- If $p \leq \alpha \rightarrow$ Reject H_0 - If $p > \alpha \rightarrow$ Fail to reject H_0
8 Draw the Conclusion	Write the conclusion in the context of the problem. - If H_0 is rejected $\rightarrow$ There is sufficient evidence to support the claim. - If H_0 is not rejected $\rightarrow$ There is insufficient evidence to support the claim.

REJECTION REGIONS (BY HYPOTHESIS)	
Right-tailed test ($H_1: \mu > \mu_0$)	 0 $t_{\alpha, df}$ Reject H_0 if $t_{calc} > t_{\alpha, df}$
Left-tailed test ($H_1: \mu < \mu_0$)	 $-\alpha$ $-t_{\alpha, df}$ 0 Reject H_0 if $t_{calc} < -t_{\alpha, df}$
Two-tailed test ($H_1: \mu \neq \mu_0$)	 $\frac{\alpha}{2}$ $-t_{\alpha/2, df}$ 0 $t_{\alpha/2, df}$ Reject H_0 if $t_{calc} > t_{\alpha/2, df}$

SUMMARY OF STEPS

1. State H_0 and H_1
2. Choose α
3. Check conditions
4. Compute $t = \frac{\bar{x} - \mu_0}{s/\sqrt{n}}$
5. Find $df = n - 1$
6. Find P-value (using t-table or software)
7. Make decision (compare p with α)
8. Draw conclusion in context

Note: For large samples ($n \geq 30$), t-test is approximately the same as z-test.

t-test table

cum. prob	$t_{.50}$	$t_{.75}$	$t_{.80}$	$t_{.85}$	$t_{.90}$	$t_{.95}$	$t_{.975}$	$t_{.99}$	$t_{.995}$	$t_{.999}$	$t_{.9995}$
one-tail	0.50	0.25	0.20	0.15	0.10	0.05	0.025	0.01	0.005	0.001	0.0005
two-tails	1.00	0.50	0.40	0.30	0.20	0.10	0.05	0.02	0.01	0.002	0.001
df											
1	0.000	1.000	1.376	1.963	3.078	6.314	12.71	31.82	63.66	318.31	636.62
2	0.000	0.816	1.061	1.386	1.886	2.920	4.303	6.965	9.925	22.327	31.599
3	0.000	0.765	0.978	1.250	1.638	2.353	3.182	4.541	5.841	10.215	12.924
4	0.000	0.741	0.941	1.190	1.533	2.132	2.776	3.747	4.604	7.173	8.610
5	0.000	0.727	0.920	1.156	1.476	2.015	2.571	3.365	4.032	5.893	6.869
6	0.000	0.718	0.906	1.134	1.440	1.943	2.447	3.143	3.707	5.208	5.959
7	0.000	0.711	0.896	1.119	1.415	1.895	2.365	2.998	3.499	4.785	5.408
8	0.000	0.706	0.889	1.108	1.397	1.860	2.306	2.896	3.355	4.501	5.041
9	0.000	0.703	0.883	1.100	1.383	1.833	2.262	2.821	3.250	4.297	4.781
10	0.000	0.700	0.879	1.093	1.372	1.812	2.228	2.764	3.169	4.144	4.587
11	0.000	0.697	0.876	1.088	1.363	1.796	2.201	2.718	3.106	4.025	4.437
12	0.000	0.695	0.873	1.083	1.356	1.782	2.179	2.681	3.055	3.930	4.318
13	0.000	0.694	0.870	1.079	1.350	1.771	2.160	2.650	3.012	3.852	4.221
14	0.000	0.692	0.868	1.076	1.345	1.761	2.145	2.624	2.977	3.787	4.140
15	0.000	0.691	0.866	1.074	1.341	1.753	2.131	2.602	2.947	3.733	4.073
16	0.000	0.690	0.865	1.071	1.337	1.746	2.120	2.583	2.921	3.686	4.015
17	0.000	0.689	0.863	1.069	1.333	1.740	2.110	2.567	2.898	3.646	3.965
18	0.000	0.688	0.862	1.067	1.330	1.734	2.101	2.552	2.878	3.610	3.922
19	0.000	0.688	0.861	1.066	1.328	1.729	2.093	2.539	2.861	3.579	3.883
20	0.000	0.687	0.860	1.064	1.325	1.725	2.086	2.528	2.845	3.552	3.850
21	0.000	0.686	0.859	1.063	1.323	1.721	2.080	2.518	2.831	3.527	3.819
22	0.000	0.686	0.858	1.061	1.321	1.717	2.074	2.508	2.819	3.505	3.792
23	0.000	0.685	0.858	1.060	1.319	1.714	2.069	2.500	2.807	3.485	3.768
24	0.000	0.685	0.857	1.059	1.318	1.711	2.064	2.492	2.797	3.467	3.745
25	0.000	0.684	0.856	1.058	1.316	1.708	2.060	2.485	2.787	3.450	3.725
26	0.000	0.684	0.856	1.058	1.315	1.706	2.056	2.479	2.779	3.435	3.707
27	0.000	0.684	0.855	1.057	1.314	1.703	2.052	2.473	2.771	3.421	3.690
28	0.000	0.683	0.855	1.056	1.313	1.701	2.048	2.467	2.763	3.408	3.674
29	0.000	0.683	0.854	1.055	1.311	1.699	2.045	2.462	2.756	3.396	3.659
30	0.000	0.683	0.854	1.055	1.310	1.697	2.042	2.457	2.750	3.385	3.646
40	0.000	0.681	0.851	1.050	1.303	1.684	2.021	2.423	2.704	3.307	3.551
60	0.000	0.679	0.848	1.045	1.296	1.671	2.000	2.390	2.660	3.232	3.460
80	0.000	0.678	0.846	1.043	1.292	1.664	1.990	2.374	2.639	3.195	3.416
100	0.000	0.677	0.845	1.042	1.290	1.660	1.984	2.364	2.626	3.174	3.390
1000	0.000	0.675	0.842	1.037	1.282	1.646	1.962	2.330	2.581	3.098	3.300
Z	0.000	0.674	0.842	1.036	1.282	1.645	1.960	2.326	2.576	3.090	3.291
	0%	50%	60%	70%	80%	90%	95%	98%	99%	99.8%	99.9%
	Confidence Level										

► Matrix- collection of vectors

Matrices are used in data science to represent datasets and perform efficient mathematical computations in machine learning, image processing, and dimensionality reduction techniques such as PCA.

A **matrix** is a rectangular arrangement of numbers in rows and columns. In data science, matrices are used to store and process data efficiently.

Main Uses:

- 1. Representing Data**
 - Rows represent observations (records).
 - Columns represent features (variables).
 - Example: A dataset containing age, salary, and experience can be stored as a matrix.
- 2. Machine Learning**
 - Algorithms such as Linear Regression, Logistic Regression, and Neural Networks perform calculations using matrices.
 - Matrix operations make computations faster and more efficient.
- 3. Image Processing**
 - Images are stored as matrices of pixel values.
 - Each pixel's intensity or color value is represented in a matrix.
- 4. Dimensionality Reduction**
 - Techniques like Principal Component Analysis (PCA) use matrices, eigenvalues, and eigenvectors to reduce the number of features while preserving important information.
- 5. Data Transformations**
 - Scaling, rotation, and other mathematical transformations are performed using matrices.

"Principal Component Analysis (PCA) is a dimensionality reduction technique that converts a large set of correlated variables into a smaller set of uncorrelated variables called principal components while preserving most of the variance present in the original data."

Why Data Science Uses Them

- **PCA (Principal Component Analysis)** uses eigenvalues and eigenvectors to reduce dimensions.
- Image compression.
- Face recognition.
- Recommendation systems.
- Feature extraction.

In PCA:

- Eigenvectors → directions of maximum variance.

- Eigenvalues → how much variance exists in those directions.

An eigenvector is a vector whose direction does not change after a matrix transformation. An eigenvalue is the scalar that tells how much the eigenvector is stretched or compressed. They are commonly used in PCA and machine learning."

► Differentiation

Differentiation is the process of finding the **rate at which a function changes** with respect to a variable.

-Differentiation is a calculus technique used to measure the rate of change of a function. In data science, it is mainly used to calculate gradients and optimize machine learning models by minimizing errors.

Why Data Scientists Learn Differentiation

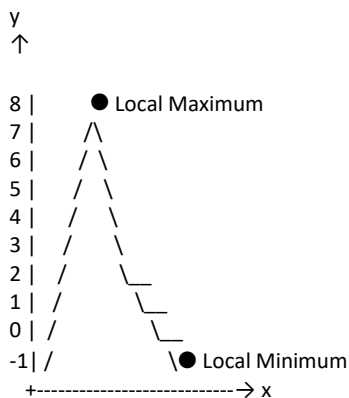
- To understand **Gradient Descent**
- To minimize **loss/error functions**
- To train **Machine Learning** and **Neural Network** models
- To understand optimization techniques

Optimization

Optimization means finding the **maximum** or **minimum** value of a function.

In Data Science, we usually want to **minimize the error (loss)** of a machine learning model.

- Optimization is the process of finding the maximum or minimum value of a function. In machine learning, we usually minimize the loss function. Graphically, optimization means finding the lowest point (minimum) or highest point (maximum) on a curve.



Slopes at Different Points

- Before the maximum point → **Positive slope (+)**
 - Graph is increasing.
- At the maximum point → **Slope = 0**
 - Horizontal tangent.
- Between maximum and minimum → **Negative slope (-)**
 - Graph is decreasing.
- At the minimum point → **Slope = 0**
 - Horizontal tangent.
- After the minimum point → **Positive slope (+)**
 - Graph is increasing again.

Optimization Rule

1. Find the derivative.
2. Set derivative equal to zero.
3. Solve for critical points.
4. Check whether each point is a maximum or minimum.

► **Probability**

Probability is the measure of how likely an event is to occur, ranging from 0 (impossible) to 1 (certain).

The basic formula is:

$$P(E) = \frac{\text{Number of favorable outcomes}}{\text{Total number of possible outcomes}}$$

Examples

- Tossing a fair coin:
 - Probability of Heads = $1/2 = 0.5$
- Drawing an Ace from a standard 52-card deck:
 - Probability = $4/52 = 1/13$

Law of large numbers

The Law of Large Numbers states that:

As the number of trials or observations increases, the sample average gets closer and closer to the true population mean (expected value).

Example:

Suppose you toss a fair coin.

- 10 tosses → Heads might appear 7 times → Probability = $7/10 = 0.7$
- 100 tosses → Heads might appear 52 times → Probability = $52/100 = 0.52$
- 1000 tosses → Heads might appear 498 times → Probability = $498/1000 = 0.498$

As the number of tosses increases, the experimental probability approaches the true probability of **0.5**.

Mutually Exclusive Events

Two events are called **mutually exclusive** if they **cannot occur at the same time**.

Example 1: Rolling a Die

Let:

- A = Getting an even number = {2, 4, 6}
- B = Getting an odd number = {1, 3, 5}

A and B cannot happen together in a single roll, so they are mutually exclusive.

Mutually Exclusive ≠ Independent

- Mutually Exclusive: Cannot happen together.
- Independent: Occurrence of one does not affect the other.

Conditional Probability

Conditional probability is the probability of an event occurring **given that another event has already occurred**.

conditional probability: the probability of event A occurring given that B has occurred which we represent with the following formula:

$$P(A|B) = \frac{P(A \cap B)}{P(B)}$$

From <<https://www.linkedin.com/learning/machine-learning-foundations-probability/articles/glossary-of-terms?autoSkip=true&resume=false&u=78898396>>

► **dependent events:** events where the occurrence of one affects the probability of the other

- Example: In image classification, if an image contains “wheels,” the probability it contains a “car” is higher. The presence of wheels increases the chance of a car being present.

From <<https://www.linkedin.com/learning/machine-learning-foundations-probability/articles/glossary-of-terms?autoSkip=true&resume=false&u=78898396>>

Q &A

09 June 2026 22:01

Q4. A company claims that the average website load time is 2 seconds. A sample of 50 users shows an average load time of 2.3 seconds. The population standard deviation is 0.5 seconds. Test at $\alpha = 0.05$ whether the load time has changed.

Given:

- $\mu = 2$ seconds
- $\bar{x} = 2.3$ seconds
- $\sigma = 0.5$ seconds
- $n = 50$
- $\alpha = 0.05$

Step 1: Hypotheses

$$H_0: \mu = 2$$

$$H_1: \mu \neq 2$$

Step 2: Calculate Z-value

$$z = \frac{\bar{x} - \mu}{\sigma / \sqrt{n}}$$

$$z = \frac{2.3 - 2}{0.5 / \sqrt{50}}$$

$$z = \frac{0.3}{0.0707}$$

$$z = 4.24$$

Step 3: Critical Value

For a two-tailed test at $\alpha = 0.05$,

$$z_{critical} = \pm 1.96$$

Step 4: Decision

$$|4.24| > 1.96$$

Therefore, **Reject H_0** .

Conclusion

At the 5% significance level, there is sufficient evidence to conclude that the **average website load time has changed from 2 seconds**.

Answer: Reject H_0 . The average website load time is significantly different from 2 seconds. ☒

Numpy

15 June 2026 12:04

Numpy:

Python library used for working with arrays

Numpy stands for Numerical python

Used for multidimensional array operations

Various other libraries like pandas,matplotlib and scikit learn are built on top of this amazing library

NumPy (Numerical Python) is a Python library used for numerical and mathematical computations. It provides support for large, multi-dimensional arrays and matrices along with various mathematical functions.

import numpy as np

In Python, **as** is used to give an **alias (short name)** to a module, package, or object when importing

Installation command- pip install numpy

Arrays

One-dimension- vector

Two dimension - matrix

Numpy arrays are called ndarray or N-dimensional arrays and they store elements of the same type and size

Numpy is regularly applied:

- Data manipulation and analysis: for data cleaning,transformation and aggregation
- Scientific computing:Handled advanced mathematical operations such as matrix multiplication,eigen values
- Machine learning:
- Signal and image processing

Creating Arrays

np.array

Zeros:np.zeros((2,3))

np.empty

In Data Science, np.empty() is used to allocate memory for large arrays when the values will be assigned later. It improves performance because NumPy does not initialize the array elements.

Functions:

Array Creation

```
import numpy as np
```

```
np.array([1, 2, 3])
np.zeros((2, 3))
np.ones((3, 3))
np.empty((2, 2))
np.arange(0, 10, 2)
np.linspace(0, 1, 5)
np.eye(3)
np.random.rand(2, 3)
np.random.randint(1, 10, (2, 2))
```

2. Array Inspection

```
a.shape    # Dimensions
a.ndim     # Number of dimensions
a.size     # Total elements
a.dtype    # Data type
```

3. Array Reshaping

```
a.reshape(2, 3)
a.flatten()
a.ravel()
a.T          # Transpose
```

4. Mathematical Operations

```
np.add(a, b)
np.subtract(a, b)
np.multiply(a, b)
np.divide(a, b)
np.power(a, 2)
np.sqrt(a)
np.exp(a)
np.log(a)
```

5. Statistical Functions

```
np.mean(a)
np.median(a)
np.std(a)
np.var(a)
np.min(a)
np.max(a)
np.sum(a)
np.cumsum(a)
```

6. Aggregation Along Axis

```
np.sum(a, axis=0) # Column-wise
np.sum(a, axis=1) # Row-wise
```

```
np.mean(a, axis=0)
np.max(a, axis=1)
```

7. Searching & Sorting

```
np.sort(a)
np.argsort(a)
np.argmax(a)
np.argmin(a)
np.where(a > 5)
```

8. Joining Arrays

```
np.concatenate((a, b))
np.vstack((a, b))
np.hstack((a, b))
```

9. Splitting Arrays

```
np.split(a, 2)
np.vsplit(a, 2)
np.hsplit(a, 2)
```

10. Linear Algebra

```
np.dot(a, b)
```

```
np.matmul(a, b)
np.linalg.inv(a)
np.linalg.det(a)
np.linalg.eig(a)
```

PANDAS

- Open source library built on top of Numpy library
- Package that offers various data structures and operations for manipulating **tabular** data,time series-eg: temperature,stock market ,and many types of data.
- Popular for importing,analyzing,cleaning,exploring and manipulating data
- Provides series 1D and DataFrame 2D structures

-> import panda as pd

Advantages:

- > Data from different resources can be loaded
- > Easy handling of missing data(represented as NaN)
- > Size mutability: columns can be inserted/deleted from DataFrame
- > Data set merging and joining
- > Allows us to analyzing big data and make conclusions based on statistical theories efficiently
- >Pandas can clean messsy data sets,make them readable and relevant
- >Relevant data is very important in data science

Series- one column-ID table

`Pd.series([10,20,30])`

Data frame- 2D table

`Data={"Name":["anu","meena"],"Age":[23,30]}`

`Df=pd.DataFrame(data)`

► Some commonly used **Pandas functions** grouped by category:

1. Reading Data

- `pd.read_csv()`
- `pd.read_excel()`
- `pd.read_json()`

Pandas

15 June 2026 15:13

PANDAS

- Open source library built on top of Numpy library
- Package that offers various data structures and operations for manipulating **tabular** data,time series-eg: temperature,stock market ,and many types of data.
- Popular for importing,analyzing,cleaning,exploring and manipulating data
- Provides series 1D and DataFrame 2D structures

-> import panda as pd

Advantages:

- > Data from different resources can be loaded
- > Easy handling of missing data(represented as NaN)
- > Size mutability: columns can be inserted/deleted from DataFrame
- > Data set merging and joining
- > Allows us to analyzing big data and make conclusions based on statistical theories efficiently
- >Pandas can clean messy data sets,make them readable and relevant
- >Relevant data is very important in data science

Series- one column-ID table

`Pd.series([10,20,30])`

Data frame- 2D table

`Data={"Name":["anu","meena"],"Age":[23,30]}`

`Df=pd.DataFrame(data)`

► In Pandas, Series and DataFrame are the two main data structures.

Series

One-dimensional (1D)

Contains a single column of data

Similar to a Python list or NumPy array with labels

Has an index and values

Created using `pd.Series()`

DataFrame

Two-dimensional (2D)

Contains multiple columns of data

Similar to an Excel sheet or SQL table

Has row index and column names

Created using `pd.DataFrame()`

1. Series

A **Series** is a one-dimensional labeled array.

Example:

```
import pandas as pd
```

```
s = pd.Series([10, 20, 30, 40])
print(s)
```

Output:

```
0    10
1    20
2    30
3    40
dtype: int64
```

You can also specify custom indexes:

```
s = pd.Series([90, 85, 95], index=["John", "Alice", "Bob"])
print(s)
```

Output:

```
John    90
Alice   85
Bob     95
dtype: int64
```

2. DataFrame

A **DataFrame** is a two-dimensional table consisting of rows and columns.

Example:

```
import pandas as pd
```

```
data = {
    "Name": ["John", "Alice", "Bob"],
    "Age": [22, 21, 23],
    "Marks": [90, 85, 95]
}
```

```
df = pd.DataFrame(data)
print(df)
```

Output:

```
   Name  Age  Marks
0  John   22    90
1  Alice  21    85
2   Bob   23    95
```

Relationship between Series and DataFrame

A **DataFrame** is a collection of **Series**.

► Some commonly used **Pandas functions** grouped by category:

1. Reading Data:

1. `pd.read_csv()`
2. `pd.read_excel()`
3. `pd.read_json()`

2. Viewing Data:

1. `df.head()` # First 5 rows
2. `df.tail()` # Last 5 rows
3. `df.info()` # DataFrame information
4. `df.describe()` # Statistical summary
5. `df.shape` # Rows and columns
6. `df.columns` # Column names

3. Selecting data:

1. `df['Name']` # Single column
2. `df[['Name','Age']]` # Multiple columns
3. `df.loc[0]` # Row by label
4. `df.iloc[0]` # Row by index

4.. Filtering:

1. `df[df['Age'] > 25]`
2. `df[df['City'] == 'Delhi']`

5. Sorting

```
df.sort_values('Age')  
df.sort_values('Age', ascending=False)
```

6. Missing Values

```
df.isnull()  
df.isnull().sum()  
df.dropna()  
df.fillna(0)
```

7. Aggregation

```
df['Salary'].mean()  
df['Salary'].sum()  
df['Salary'].max()  
df['Salary'].min()  
df['Salary'].count()
```

8. Grouping

```
df.groupby('Department')['Salary'].mean()  
df.groupby('City').count()
```

9. Unique Values

```
df['City'].unique()  
df['City'].nunique()
```

10. Column Operations

```
df['Bonus'] = df['Salary'] * 0.1  
df.rename(columns={'Name':'Employee_Name'})  
df.drop('Bonus', axis=1)
```

11. Merging & Joining

```
pd.merge(df1, df2, on='ID')  
df1.join(df2)
```

12. Duplicate Handling

```
df.duplicated()  
df.drop_duplicates()
```

13. Value Counts

```
df['City'].value_counts()
```

14. Correlation

```
df.corr()
```

.map() function in Pandas

The `.map()` function is used to **transform or replace values in a Series (single column)**

○ Query()

The `query()` method is used to **filter rows based on a condition**. It is an alternative to Boolean indexing and often makes **conditions easier to read**.

Eg:

-Salary greater than 65000

Using Boolean indexing:

✓ `dd[dd["salary"] > 65000]`

Using query():

✓ `dd.query("salary > 65000")`

From <https://colab.research.google.com/drive/1o1moYtZAOP_q7vK33qwzdAx7D-6RU5Tk#scrollTo=5J323_DnavPM>

Database and SQL

19 June 2026 09:38

Database = A structured collection of data that is stored electronically and can be easily accessed, managed, updated, and retrieved.

Used for:

- Improve business processes
- Keep track of customers
- Store personal data
- Keep a record of music or book collections
- Finances
- Healthcare
- Weather
- Government Organizations
- Based on historical data, can manage data, retrieve etc

Types of databases:

- Relational databases-structured database
- Object-oriented databases
- Distributed data base
- Data warehouses
- NoSQL databases- no structured database

SQL:

- Structured Query Language
- Access and manipulate databases
- Eg: MySQL, Oracle

Importance of SQL in Data Analysis and Data Science:

What can SQL DO?

- Can execute queries against a database
- Can retrieve data from a database
- Can insert records in a database
- Can update records in a database
- Can delete records from a database
- Can create new databases
- Can create new tables in a database

Important terminologies:

- Database Tables are referred to as *relations*
- The columns of a table are referred to as attributes or fields
- Rows are referred to as tuples or records
- The number of attributes in the relations is known as degree of the relations
- The number of tuples in a relation is known as cardinality
- Entity is an object that exists in real-world- single table or a particular entity

Different types of keys:

- Primary key- can't allow null values- use for relations

- Foreign key
- Composite key
- Unique key-same as foreign ,but can allow 1 null value

Datatype:

- Int
- Varchar
- Char
- Boolean
- Float
- Date
- Datetime

Database languages:

- Data definition Language
- Data manipulation Language
- Data control Language
- Transaction control Language

Data definition Language-use at the time of data structure creating or changing

- Create
- Alter
- Drop

Data Manipulation Language:use to store,modify,retrieve,delete and update data in database

- Select
- Insert
- Update
- Delete

Data Control Language: used for access permission controlling

- Grant
- Revoke

Transaction Control Language:

Deals with transaction within a database

- Commit
- Rollback
- Savepoint

Sql commands:

1. SELECT
2. UPDATE
3. DELETE
4. INSERT INTO- The **INSERT INTO** command is used to add new records (rows) into a table.
5. CREATE DATABASE
6. ALTER DATABASE- modifies a database
7. CREATE TABLE
8. ALTER TABLE - modifies a table
9. DROP TABLE - whole table deletion
10. TRUNCATE TABLE - maintain the structure,but delete the contents in the table

Samp

First database create->then table->insert into()-values

- select* from table:
Show all contents in table
- Where- condition
- DESC - for get in descending order
- limit2- get 2 rows
- distinct-get particular values
- Order by-use for data sort
- Groupby-use for grouping

Alter table-structure change

ALTER:

- ALTER → table structure maattunnu (column add, rename, datatype change...)
- UPDATE → table-ilulla data maattunnu

Eg:

```
ALTER TABLE Patients  
RENAME COLUMN Age TO patient_age;
```

Eg:

```
ALTER TABLE Patients  
ADD Email VARCHAR(50);
```

Eg:

```
ALTER TABLE Patients  
DROP COLUMN Email;
```

Eg:

```
ALTER TABLE Patients  
MODIFY Contact_No BIGINT;
```

Eg:

```
ALTER TABLE Patients  
RENAME TO Patient_Details;    -table rename
```

Eg:

```
ALTER TABLE Patients  
MODIFY patient_id INT AUTO_INCREMENT;
```

UPDATE:

UPDATE ... WHERE ...

UPDATE ... IN (...)

UPDATE ... LIKE ...

UPDATE ... IS NULL

UPDATE multiple columns

->>IN Operator is used to check multiple values in a single condition.

Instead of writing:

```
SELECT * FROM Patients  
WHERE patient_id = 113
```

OR patient_id = 123
OR patient_id = 145;

You can write:

```
SELECT * FROM Patients  
WHERE patient_id IN (113, 123, 145);
```

Example 1: SELECT

```
SELECT * FROM Doctors  
WHERE DoctorID IN (2345, 2346);
```

o/p Shows doctors whose ID is either 2345 or 2346.

-->> **LIKE is used for pattern matching in SQL.**

% = Any number of characters

Starts with A

```
SELECT * FROM Patients  
WHERE patient_name LIKE 'A%';  
Matches:
```

O/p: Arjun
Anu
Amrutha

Ends with n

```
SELECT * FROM Patients  
WHERE patient_name LIKE '%n';  
Matches:
```

O/p :Arjun

Contains "ru"

```
SELECT * FROM Patients  
WHERE patient_name LIKE '%ru%';  
Matches:
```

O/p: Arjun

-->>**ORDER BY is used to sort the output**

ORDER BY is used to **sort the records in a table** in ascending or descending order.

Eg:

Sort by Age (Descending)

```
SELECT * FROM Patients  
ORDER BY patient_age DESC;
```

Output: Highest age to lowest age

Eg:

Sort by Age (Ascending)

```
SELECT * FROM Patients  
ORDER BY patient_age ASC;
```


Output: Lowest age to highest age

-->>**GROUP BY** is used to group rows that have the same value in a column and usually works with aggregate functions like **COUNT()**, **SUM()**, **AVG()**, **MAX()**, **MIN()**.

Eg:

```
SELECT Doctor_name, COUNT(*)
FROM Doctors
GROUP BY Doctor_name;
```

Output:

```
Arun    1
Midhun  1
Sreejith 1
```

Eg:

```
SELECT Specialization, COUNT(*) AS total
FROM Doctors
GROUP BY Specialization;
```

Specialization total

```
Cardio      3
Ortho       2
```

From <<https://chatgpt.com/c/6a360f3e-a7f8-83ee-ac21-60a9fbc43026>>

- Specialization → shows the specialization name.
- COUNT(*) → counts the number of rows (doctors).
- AS total → gives the count column the name **total**.

-->>**WHERE-applying conditions**

Filter rows.

Eg:SELECT * FROM Patients
WHERE Age > 25;

Applying where using,

-->>Comparison operator

1.<>

(Not Equal)

Eg:
SELECT * FROM Patients
WHERE Gender <> 'Male';

-->>BETWEEN

Range of values.

Eg:

```
SELECT * FROM Patients
WHERE Age BETWEEN 20 AND 30;
```

Eg:

```
SELECT * FROM Appointments
WHERE AppointmentDate
```

BETWEEN '2026-06-20' AND '2026-06-22';

-->>**DISTINCT**

Remove duplicates.

Eg:

```
SELECT DISTINCT Gender
FROM Patients;
```

Differences in terms

Eg:MAX(Specialization)

→ Alphabetically largest text- here in specialization column,which text is largest,means length of text calculate here

MAX(patient_age)

→ Largest age value

COUNT(*)

→ Number of rows

GROUP BY + COUNT(*) + ORDER BY DESC

→ Most frequent value (Data Science-il valare common aanu)

-->>**CASE WHEN = SQL's IF-ELSE statement.** It is very common in Data Analytics and Data Science interviews.

Syntax:

```
SELECT column_name,
CASE
    WHEN condition1 THEN result1
    WHEN condition2 THEN result2
    ELSE result3
END AS new_column
FROM table_name;
```

Eg:

```
SELECT patient_name,
patient_age,
CASE
    WHEN patient_age < 18 THEN 'Child'
    WHEN patient_age BETWEEN 18 AND 59 THEN 'Adult'
    ELSE 'Senior Citizen'
END AS Age_Group
FROM patients;
```

-->>**HAVING**

- **WHERE** → filter rows
- **HAVING** → filter groups

HAVING **GROUP BY**-nu shesham aggregated data filter cheyyan use cheyyunnu.

WHERE

Individual rows filter cheyyum

GROUP BY-nu munpu work cheyyum

Aggregate functions use cheyyan pattilla

HAVING

Groups filter cheyyum

GROUP BY-nu shesham work cheyyum

Aggregate functions use cheyyam

Syntax:

```
SELECT column_name, COUNT(*)  
FROM table_name  
GROUP BY column_name  
HAVING COUNT(*) > 1;
```

From <<https://chatgpt.com/c/6a393a6f-b1cc-83ee-b697-7726ae93673f>>

⇒ JOINS

A **JOIN** is used to combine rows from two or more tables based on a related column.

1. INNER JOIN

Returns only matching rows from both tables
-common informations fetch

2. LEFT JOIN (LEFT OUTER JOIN)-take all informations in first table and take common things in right also

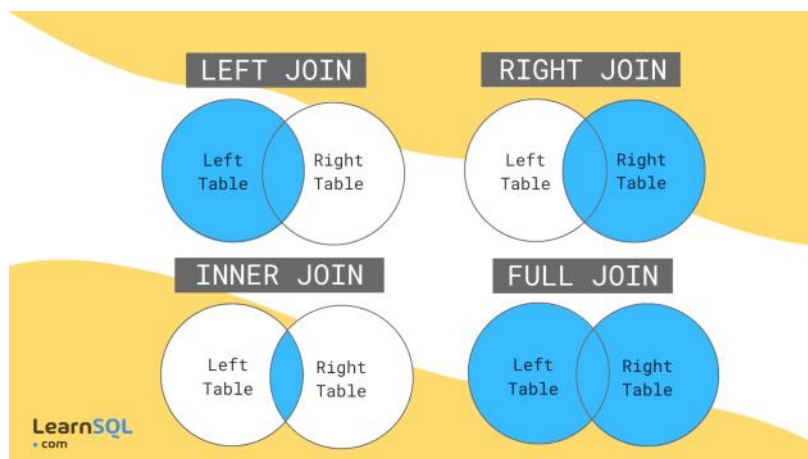
Returns all rows from the left table and matching rows from the right table.

3. RIGHT JOIN (RIGHT OUTER JOIN)

Returns all rows from the right table and matching rows from the left table.

4. FULL JOIN (FULL OUTER JOIN)

Returns all rows from both tables, matched where possible.



➤ INNER JOIN SYNTAX

```
SELECT column1, column2, ...  
FROM table1  
INNER JOIN table2  
ON table1.common_column = table2.common_column;
```

➤ LEFT JOIN SYNTAX

```
SELECT column1, column 2  
From table 1  
LEFT JOIN table2  
ON table1.id =table2.id
```

➤ RIGHT JOIN SYNTAX

SELECT column1.column2
From Table1
RIGHT JOIN table2
ON table1.id =table2.id

➤ OUTER JOIN SYNTAX

SELECT column1.column2
From table1
OUTER JOIN table 2
ON table.id=table2.id

-->>**RANK()** is a window function that assigns a rank to each row based on the specified order. Rows with the same value receive the same rank, and the next rank is skipped.

-->>**COUNT() OVER()** calculates the count as a window function and returns the count for each row without collapsing rows like GROUP BY does.

Difference b/w count and count()over()

4. Count - with GROUP BY

Eg:
SELECT Department,
COUNT(*) AS DeptCount
FROM Members
GROUP BY Department;

Result:

Department	DeptCount
CSE	2
ECE	2
EEE	1

From <<https://chatgpt.com/c/6a38b59d-a5b4-83ee-a70f-ec7681b976d4>>

5. COUNT() OVER() - without GROUP BY

SELECT student_name,
Department,
COUNT(*) OVER(PARTITION BY Department) As DeptCount
FROM Members;

student_name	Department	DeptCount
Lekshmi	CSE	2
Anu	CSE	2
Meenu	ECE	2
Anju	ECE	2
Sera	EEE	1

From <<https://chatgpt.com/c/6a38b59d-a5b4-83ee-a70f-ec7681b976d4>>

Result keeps **all rows** and shows the count beside each row. This is the main difference.

► Window Functions in SQL

Window Functions perform calculations across a set of rows related to the current row **without grouping the rows into a single result**.

Syntax:

```
function_name() OVER(  
    [PARTITION BY column]  
    [ORDER BY column]  
)
```

1. ROW_NUMBER() - Gives a unique number to each row.

Eg:

```
SELECT student_name,  
       ROW_NUMBER() OVER(ORDER BY student_name) AS RowNo  
FROM Members;
```

Output:

student_name	RowNo
Anju	1
Anu	2
Lekshmi	3
Meenu	4

2. RANK() - Same values get same rank, next rank is skipped.

Eg:

```
SELECT student_name,  
       RANK() OVER(ORDER BY Marks DESC) AS RankNo  
FROM Students;
```

Example:

Marks	Rank
100	1
95	2
95	2
90	4

3. DENSE_RANK() - Same values get same rank, but no skipping.

Eg:

```
SELECT student_name,  
       DENSE_RANK() OVER(ORDER BY Marks DESC) AS RankNo  
FROM Students;
```

Marks	Dense Rank
100	1
95	2
95	2
90	3

4. COUNT() OVER() - Counts rows without grouping.

Eg:
SELECT student_name,
COUNT(*) OVER() AS TotalMembers
FROM Members;

5. SUM() OVER()- Running total.

Eg:
SELECT SaleDate,
Amount,
SUM(Amount) OVER(ORDER BY SaleDate) AS RunningTotal
FROM Sales;

6. AVG() OVER() - Average value for each row.

Eg:
SELECT student_name,
Marks,
AVG(Marks) OVER() AS AvgMarks
FROM Students;

7. MIN() OVER() and MAX() OVER()

Eg:
SELECT student_name,
Marks,
MIN(Marks) OVER() AS MinMarks,
MAX(Marks) OVER() AS MaxMarks
FROM Students;

8. LAG() - Gets the previous row value.

Eg:
SELECT SaleDate,
Amount,
LAG(Amount) OVER(ORDER BY SaleDate) AS PreviousSale
FROM Sales;

9. LEAD() - Gets the next row value.

Eg:
SELECT SaleDate,
Amount,
LEAD(Amount) OVER(ORDER BY SaleDate) AS NextSale
FROM Sales;

Reading Data from various sources

24 June 2026 09:46

Data Source	Function	Example
CSV file	<code>pd.read_csv()</code>	<code>df = pd.read_csv("employees.csv")</code>
Excel file	<code>pd.read_excel()</code>	<code>df = pd.read_excel("employees.xlsx")</code>
JSON file	<code>pd.read_json()</code>	<code>df = pd.read_json("data.json")</code>
SQL Database	<code>pd.read_sql()</code>	<code>df = pd.read_sql("SELECT * FROM employee", connection)</code>
HTML Table	<code>pd.read_html()</code>	<code>tables = pd.read_html("https://example.com")</code>
XML file	<code>pd.read_xml()</code>	<code>df = pd.read_xml("data.xml")</code>
Parquet file	<code>pd.read_parquet()</code>	<code>df = pd.read_parquet("data.parquet")</code>
Clipboard	<code>pd.read_clipboard()</code>	<code>df = pd.read_clipboard()</code>
Fixed-width text	<code>pd.read_fwf()</code>	<code>df = pd.read_fwf("data.txt")</code>

From <<https://chatgpt.com/c/6a49d994-8298-83e8-b82b-7bf1863cfff3>>

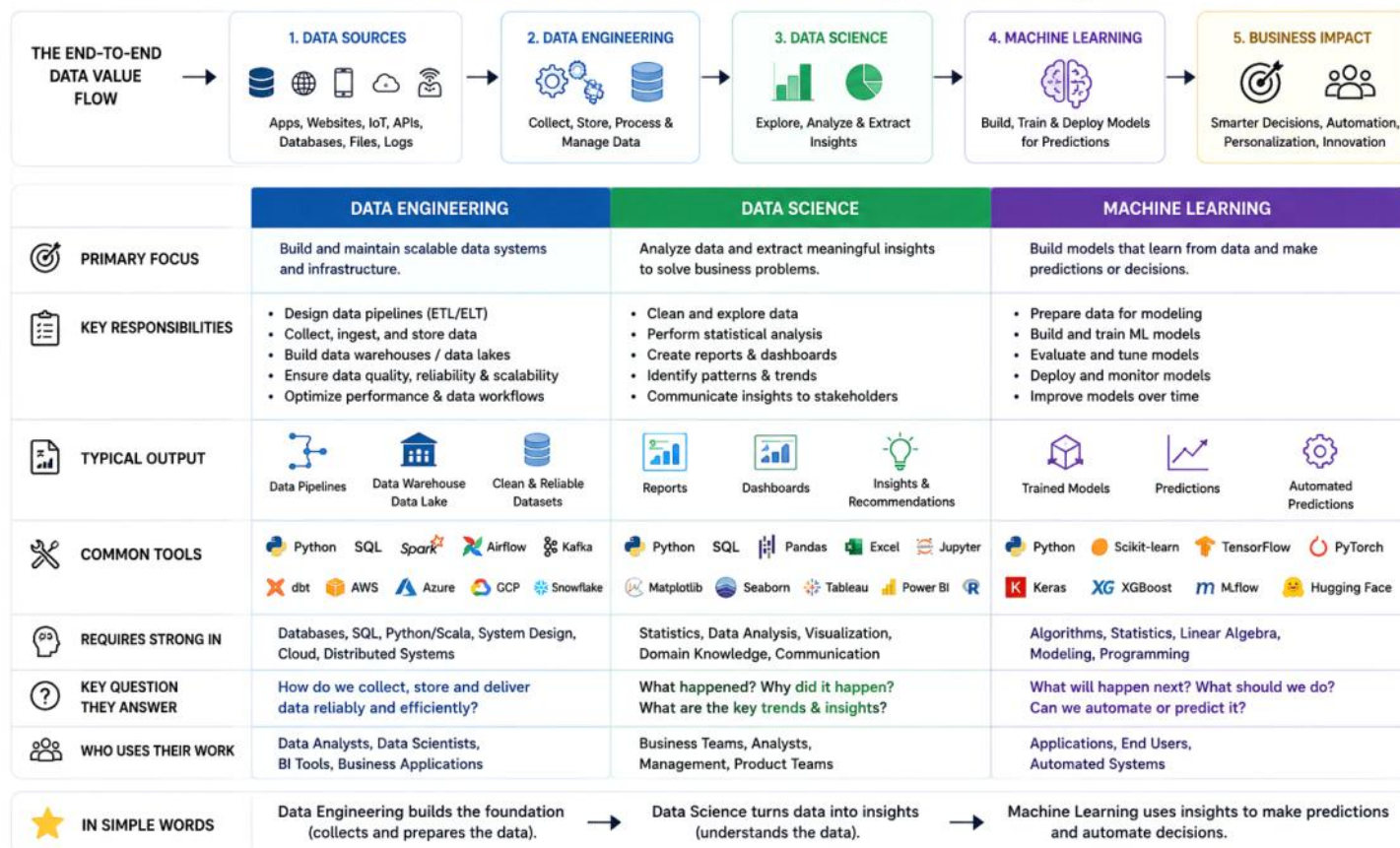
Data Engineering VS DataScience VS MachineLearning

24 June 2026 23:11

Data Engineering is the field of designing, building, and maintaining systems that collect, store, process, and deliver data for analytics, machine learning, and business applications.

DATA ENGINEERING vs DATA SCIENCE vs MACHINE LEARNING

Different Focus. Different Tools. Same Goal: Turning Data into Intelligence.



Pipeline- Flow of Data

ETL and ELT are two methods for moving and preparing data. The main difference is when the data is transformed.

ETL (Extract → Transform → Load)

Data Sources



Extract



Transform

(Clean, filter, format, aggregate)



Load

(Data Warehouse)

Example:

- Extract customer data from a website.
- Transform: remove duplicates, fix missing values, standardize dates.
- Load the cleaned data into a data warehouse

ELT (Extract → Load → Transform)

Data Sources



Extract



Load

(Data Warehouse / Data Lake)



Transform

(SQL, Spark, dbt, etc.)

Example:

- Extract raw customer data.
- Load it directly into a cloud data warehouse.
- Transform it later using SQL or other processing tools.

ML

Framework:

Tensorflow, PyTorch, scikit_learn

MLops & Deployment:

Docker, Kubernetes,

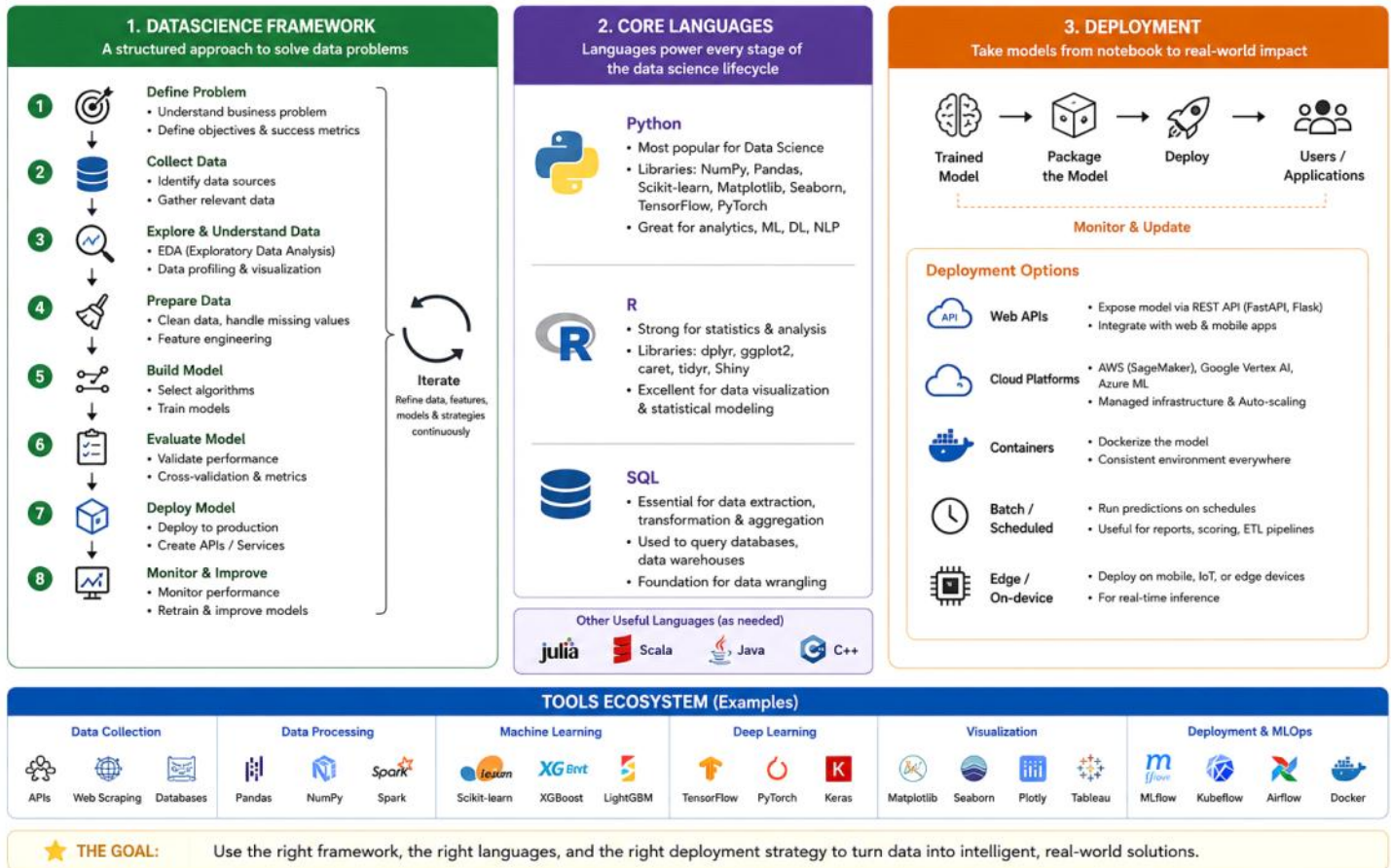
CI/CD Pipelines

Core Languages:

Python (Strong SE Fundamentals)

DATASCIENCE: FRAMEWORK, CORE LANGUAGES & DEPLOYMENT

From Problem to Production: The Data Science Lifecycle



Data Ingestion Techniques:Batch vs Streaming Data

29 June 2026 09:55

Data Ingestion:
It is the process of gathering,managing,and utilizing data efficiently.
Plays a foundational step in the data processing pipeline.
It involves the seamless

- **Batch Processing:** Processes a large amount of data at scheduled intervals.
- **Real-time Processing:** Processes data immediately as it arrives.
- **Micro-batching:** Processes small batches of data at frequent intervals, providing near real-time results.

Batch Processing

Definition:
Batch processing collects data over a period of time and processes it all at once.

- Examples:**
- Daily sales report
 - Monthly bank statement
 - Payroll processing

- Advantages:**
- Efficient for large volumes of data
 - Cost-effective
 - Simple to manage

- Disadvantages:**
- Results are delayed
 - Not suitable for applications requiring instant updates

Real-Time Processing (Stream Processing)

Definition:
Real-time processing processes data immediately as it arrives, with little or no delay.

- Examples:**
- UPI payment notifications
 - Credit card fraud detection
 - Live GPS tracking
 - Stock market updates

- Advantages:**
- Instant results
 - Supports live monitoring and decision-making

- Disadvantages:**
- More complex to implement
 - Higher infrastructure cost

Micro-Batching

Definition:
Micro-batching processes data in small batches at short, regular intervals (e.g., every 5, 10, or 30 seconds). It provides near real-time processing.

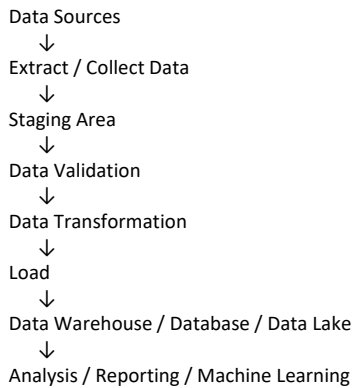
- Examples:**
- Website click analysis every 10 seconds
 - IoT sensor data processing every 30 seconds
 - Log analysis every minute

- Advantages:**
- Faster than batch processing
 - Easier to implement than true real-time processing

- Disadvantages:**
- Not true real-time
 - Small processing delay still exists

Feature	Batch Processing	Real-Time Processing	Micro-Batching
Processing Method	Large batches	One record at a time	Small batches
Speed	Slow	Very Fast	Fast
Delay	Minutes/Hours	Milliseconds/Seconds	Few Seconds
Cost	Low	High	Medium
Best For	Scheduled reports	Live applications	Near real-time analytics

The data Ingestion workflow



Staging is a temporary storage area where raw data is stored before validation, cleaning, transformation, and loading into the final destination.

Validate

Check for:

- Missing values
- Duplicate records
- Incorrect data types
- Invalid records

Transform

Prepare the data by:

- Cleaning
- Filtering
- Renaming columns
- Converting data types
- Aggregating data

Load

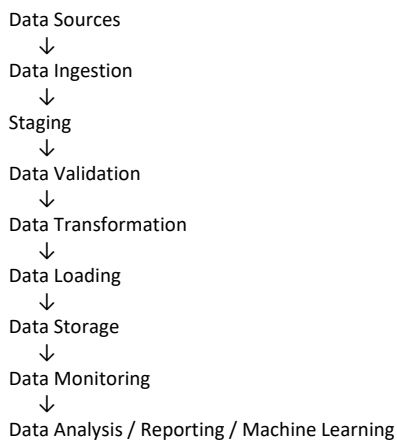
Load the processed data into:

- Database
- Data Warehouse
- Data Lake

Analysis

The stored data is used for:

- Business Intelligence (BI)
- Dashboards
- Machine Learning
- Data Analytics



Data Monitoring

Data monitoring means continuously checking the health and quality of the data.

Examples:

- Are new records arriving on time?
- Are there missing values?
- Are there duplicate records?
- Did the ETL pipeline fail?
- Is the data accurate and complete?

Data Lake

Definition:

A **Data Lake** is a centralized repository that stores **raw data** in its original format until it is needed.

Stores

- Structured data (CSV, SQL tables)
- Semi-structured data (JSON, XML)
- Unstructured data (Images, Videos, Audio, PDFs)

Example

CSV
JSON
Images
Videos
Logs
↓
Data Lake

Best for:

- Machine Learning
- Big Data Analytics
- AI Projects

Data Warehouse

Definition:

A **Data Warehouse** is a centralized repository that stores **cleaned, transformed, and structured data** for reporting and business intelligence.

Stores

- Only processed and structured data

Example

Raw Data
↓
ETL (Clean & Transform)
↓
Data Warehouse
↓
Power BI / Reports / SQL Queries

Best for:

- Business reports
- Dashboards
- SQL queries
- Decision making

► common Batch Processing tools used in Data Engineering and Data Science.

Tool	Purpose
Apache Spark	Large-scale batch data processing
Apache Hadoop (MapReduce)	Distributed batch processing of big data
Apache Hive	SQL-based batch processing on Hadoop
Apache Airflow	Schedules and automates batch workflows (orchestration)
Apache Oozie	Workflow scheduler for Hadoop batch jobs
Talend	ETL and batch data integration
Informatica PowerCenter	Enterprise ETL and batch processing
AWS Glue	Cloud ETL and batch processing
Azure Data Factory	Cloud data integration and batch pipelines
Google Cloud Dataflow	Supports both batch and stream processing

From <<https://chatgpt.com/c/6a41f6a1-a828-83e8-b37e-44353caa8d7f>>

Most Common Tools

- **Apache Spark** – Batch processing, ETL, analytics
- **Hadoop MapReduce** – Large-scale batch processing
- **Apache Airflow** – Scheduling and automating batch jobs
- **AWS Glue** – Cloud-based batch ETL
- **Azure Data Factory** – Batch data pipelines

⇒ Stream Processing (Real-Time Processing)

Stream Processing is the continuous processing of data as it is generated. Instead of waiting to collect all the data, each event or record is processed immediately or with very low latency.

Examples

- UPI payment processing
- Credit card fraud detection
- Live stock market updates
- GPS navigation
- Social media feeds
- IoT sensor monitoring

Advantages

- Provides real-time insights.
- Detects issues immediately.
- Supports instant decision-making.
- Low latency (milliseconds to seconds).

Disadvantages

- More complex to build and maintain.
- Higher infrastructure cost.
- Requires continuous monitoring.

► **Common Stream Processing Tools:**

Tool	Purpose
Apache Kafka	Distributed event streaming platform
Apache Spark Streaming	Real-time stream processing
Apache Flink	High-performance stream processing
Apache Storm	Real-time data processing
Apache Pulsar	Event streaming and messaging
Amazon Kinesis	AWS real-time data streaming
Azure Event Hubs	Microsoft Azure event streaming
Google Cloud Pub/Sub	Real-time messaging and event ingestion

From <<https://chatgpt.com/c/6a41f6a1-a828-83e8-b37e-44353caa8d7f>>

Apache Kafka

Apache Kafka is an **open-source distributed event streaming platform** used to **collect, store, and process real-time data streams**.

In simple terms:

Kafka acts like a **high-speed messaging system** that transfers data from one application to another in real time.

Why is Kafka used?

- Real-time data streaming
- High-speed data transfer
- Handling millions of events/messages
- Building data pipelines
- Connecting different applications

How Kafka Works:

Producer ----> Kafka Broker ----> Consumer
(Sends) (Stores) (Reads)

Kafka Architecture:

Producer → Topic → Broker → Consumer

- **Producer:** Sends messages to Kafka.
- **Topic:** Category where messages are stored.
- **Broker:** Kafka server that stores and manages messages.
- **Consumer:** Reads messages from Kafka.

► Main Components:

Component	Description
Producer	Sends data to Kafka.
Consumer	Reads data from Kafka.
Broker	Kafka server that stores messages.
Topic	Logical channel for messages.
Partition	Divides a topic for parallel processing.
Offset	Unique position of each message in a partition.
Consumer Group	Multiple consumers sharing the workload.

From <<https://chatept.com/c/6a4207d8-de84-83e8-a713-b33a0d79e922>>

Offset

- A unique sequential number assigned to each message in a partition.
- Tracks the position of messages.
- Helps consumers resume reading from the last processed message.

Partition

- Splits a topic into smaller parts.
- Enables parallel processing.
- Improves scalability and performance.

Consumer Group

- A group of consumers reading the same topic.
- Each partition is processed by only one consumer in the group.
- Increases processing speed and fault tolerance.

One-line Definitions

- **Producer:** Sends messages to Kafka.
- **Consumer:** Reads messages from Kafka.
- **Broker:** Kafka server that stores messages.
- **Topic:** Category/channel where messages are stored.
- **Partition:** Division of a topic for parallel processing.
- **Offset:** Unique position number of a message in a partition.
- **Consumer Group:** Multiple consumers working together to process messages.

► Eg:

Apache Kafka – E-commerce Website Example

Scenario:

A customer places an order on an e-commerce website.

Order Details:

Order ID : 101
 Customer : Rahul
 Product : iPhone 16
 Amount : ₹80,000

Step 1: Producer

The **e-commerce website** acts as the **Producer**.
 It sends the order details to Kafka.

Website
 |
 ▼
 Producer

Step 2: Topic

The Producer sends the order to the **Orders** topic.

Producer

↓
▼
Topic : Orders

Step 3: Broker

The **Kafka Broker** receives and stores the message.

Producer
↓
▼
Kafka Broker
↓
▼
Topic : Orders

Step 4: Partition & Offset

Kafka stores messages inside partitions.

Orders Topic

Partition 0

Offset 0 → Order #101

Offset 1 → Order #102

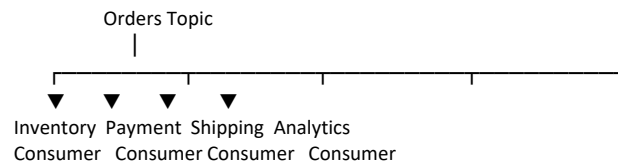
Offset 2 → Order #103

Offset 3 → Order #104

Offset: Unique position number of each message in a partition.

Step 5: Consumers

Multiple services read the same order independently.



Inventory Consumer

- Reduces product stock.

Payment Consumer

- Verifies payment.
- Generates invoice.

Shipping Consumer

- Creates shipment.
- Assigns courier.

Analytics Consumer

- Updates sales dashboard.
- Calculates revenue.

Workflow

1. Customer books a courier.
2. The **Courier App (Producer)** sends the booking event to Kafka.
3. Kafka stores the event in the **Courier_Orders** topic.
4. The topic is divided into **partitions**.
5. Each message gets an **offset**.
6. Different **consumer groups** read the same event:
 - **Pickup Service** → Assigns pickup agent.
 - **Tracking Service** → Updates package location.
 - **Delivery Service** → Assigns delivery executive.
 - **Notification Service** → Sends SMS/Email updates.

Components

- **Producer** → Courier Website/App
- **Broker** → Apache Kafka Server
- **Topic** → Courier_Orders
- **Partition** → Divides the topic for parallel processing
- **Offset** → Unique position of each message
- **Consumer Groups** → Pickup, Tracking, Delivery, Notification services

➤ ZooKeeper (Apache ZooKeeper)

- **ZooKeeper** is a centralized service that **manages and coordinates Kafka brokers** in a Kafka cluster.
- It helps Kafka keep track of brokers, topics, partitions, and leader election.

Another eg:

Banking app fraud alerts

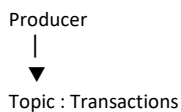
Step 1: Customer makes a transaction

Example:

Customer transfers ₹50,000
The **Banking App** acts as the **Producer**.

Step 2: Producer sends the transaction

The app sends the transaction details to the Kafka topic **Transactions**.



Step 3: Kafka stores the message

Kafka stores the transaction in the **Transactions** topic.

Transactions Topic

Offset 0 → ₹5,000
Offset 1 → ₹50,000
Offset 2 → ₹1,20,000

Step 4: Multiple Consumers process the same transaction

Fraud Detection Consumer

- Checks if the transaction is suspicious.
- Detects unusual login location or large amount.
- Flags possible fraud.

Account Service Consumer

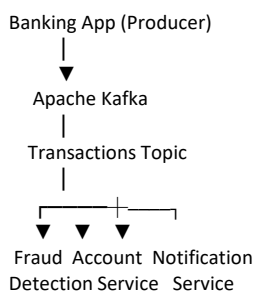
- Updates the customer's account balance.

Notification Service Consumer

- Sends an SMS, email, or push notification.

Example:

"₹50,000 debited from your account.
If this wasn't you, contact the bank immediately."



ETL vs ELT: Building Robust Data Pipelines Using Apache Airflow

30 June 2026 09:56

What is Apache Airflow?

Apache Airflow is an open-source workflow orchestration tool used to automate, schedule, and monitor data pipelines.

Why use Airflow?

- Automates ETL and ELT workflows
- Schedules pipelines (hourly, daily, weekly)
- Monitors task execution
- Handles task dependencies
- Supports retries on failure
- Easy to scale and maintain

Airflow Components

- **DAG (Directed Acyclic Graph):** Defines the workflow and task dependencies.
- **Task:** A single unit of work (e.g., extract data).
- **Operator:** Specifies what a task does (Python, SQL, Bash, etc.).
- **Scheduler:** Triggers workflows based on schedules.
- **Executor:** Runs the tasks.
- **Web UI:** Monitors pipeline status and logs.

Apache Airflow is a workflow orchestration (pipeline management) tool used to schedule, automate, and monitor data pipelines.

Used in Batching not in realtime

What Airflow does

- Schedules pipelines (e.g., every day at 2 AM)
- Executes tasks in the correct order
- Manages task dependencies
- Retries failed tasks automatically
- Monitors pipeline status through a web interface

Airflow coreconcepts:

- DAG: Directed acyclic Graph
- Task: one step in the pipeline
- Operator: (type of task: Python, Bash, SQL, etc)

1. Scheduler

The **Scheduler** is the **brain of Apache Airflow**. It checks the DAGs, determines when tasks should run, and triggers them according to the schedule and dependencies.

Functions:

- Monitors DAGs
- Schedules tasks
- Triggers task execution
- Ensures task dependencies are met

Example:

If a DAG is scheduled to run every day at **2:00 AM**, the Scheduler starts it at that time.

2. Metadata Database (*not "metascore database"*)

The **Metadata Database** stores all the information Airflow needs to manage workflows.

It stores:

- DAG information
- Task status (success, failed, running)
- Execution history
- Logs and scheduling information

Common databases used:

- SQLite (development)
- PostgreSQL (production)
- MySQL (production)

3. Web UI

The **Web UI** is Airflow's web-based interface used to monitor and manage workflows.

Functions:

- View DAGs
- Monitor task status
- Trigger DAGs manually
- View logs
- Retry failed tasks

CREATING AN AIRFLOW PROJECT:Steps:

- Dag/
- Include/
- Plugins/
- tests/
- airflow_settings.yaml
- Dockerfile
- Packages.txt
- Requirements.txt

PROJECT

02 July 2026 19:58

1. What is your data source?

The dataset is obtained from the **UCI Machine Learning Repository**. It contains hourly records of bike-sharing rentals collected during 2011 and 2012.

2. Why did you choose it?

I chose this dataset because it is a real-world dataset with a sufficient number of records (17,379 rows) and multiple features. It is ideal for learning and implementing an ETL pipeline, including data extraction, cleaning, transformation, and loading into a database.

3. What information does it contain?

The dataset contains hourly bike rental information along with weather and calendar-related features.

It includes:

- Record ID
- Date
- Season
- Year
- Month
- Hour
- Holiday
- Day of the week
- Working day
- Weather situation
- Temperature
- Feels-like temperature
- Humidity
- Wind speed
- Number of casual users
- Number of registered users
- Total bike rentals

4. Is it structured, semi-structured, or unstructured?

The dataset is **structured data** because it is organized into rows and columns with predefined data types.

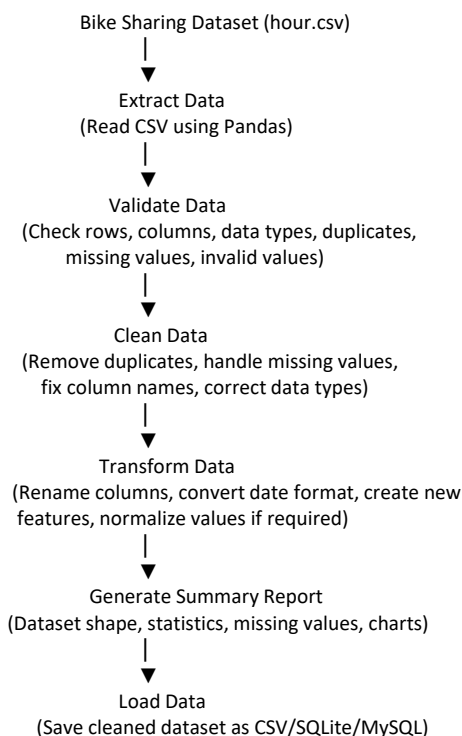
5. What format is the data?

The data is stored in **CSV (Comma-Separated Values)** format.

6. How frequently is the data updated?

This is a **historical dataset** and is **not updated regularly**. It contains hourly bike-sharing data collected during **2011 and 2012**.

Flow-Diagram



Feature Engineering and Visualization

03 July 2026 09:43

➤ Why is crucial for building successful machine learning models?

- Improves Model Accuracy-reveal hidden patterns in the data and make more accurate predictions
- Reduces overfitting- By selecting and transforming features that generalize well across different datasets
- Simplifies the Model- Reduce the complexity
- Enhances Interpretability

Real-word examples:

- E-commerce systems
- Credit Scoring
- Healthcare Predictive Models

► DataTypes:Continuous and Discrete

- Continuous Features:Features that take any value within a range,often representing measurements or quantities (eg:height,weight,temperature)

Concept	Meaning	Example from your Bike Sharing dataset
<u>Continuous Data</u>	<u>Numeric values that can take any value within a range (including decimals).</u>	<u>Temperature = 0.24, Humidity = 0.64, Wind Speed = 0.1940</u>
<u>Discrete Data</u>	<u>Countable values (integers or categories).</u>	<u>Hour = 10, Total Rentals = 172, Season = Spring</u>
<u>Normalization</u>	<u>Scaling numeric values to a common range (usually 0–1).</u>	<u>Temperature = 0.24, Humidity = 0.64</u>

From <<https://chatgpt.com/c/6a473714-67f4-83ee-bd8e-efaa59c2f676>>

For **continuous data**, there are several common transformations used in data preprocessing and feature engineering.

Transformation	Purpose	Example
Normalization (Min-Max Scaling)	Scale values to 0–1	Temperature: 20°C → 0.50
Standardization (Z-score Scaling)	Mean = 0, Standard deviation = 1	Salary → z-score
Log Transformation	Reduce skewness in highly skewed data	Income, Sales

Visualization in Continuous data:

- Histogram:
- Density plot
- Tools: Matplotlib,Seaborn

► **Techniques for handling discrete data:**

- One-Hot Encoding
- Label-Encoding

One-Hot Encoding

One-Hot Encoding is a technique used to convert **categorical (text) data into multiple binary (0 or 1) columns**. Each unique category gets its own column-create new column

Color

Red

Blue

Green

Red

From <<https://chatgpt.com/c/6a474079-90c0-83e8-9611-5d4ca986ab7c>>

After One-Hot Encoding:

Color_Blue	Color_Green	Color_Red
0	0	1
1	0	0
0	1	0
0	0	1

Label-Encoding

Label Encoding is a preprocessing technique used in machine learning to convert **categorical (text) data into numerical values** by assigning each unique category an integer.-
Replace column with numerical values

Color	Label Encoded
Red	0
Blue	1
Green	2
Red	0
Green	2

From <<https://chatgpt.com/c/6a474079-90c0-83e8-9611-5d4ca986ab7c>>

► Feature Extraction:

Feature Extraction is the process of creating or extracting **new, meaningful features** from raw data while reducing unnecessary information. It helps machine learning models learn patterns more effectively.

► Common Feature Extraction Techniques

1. Principal Component Analysis (PCA)

- Reduces the number of features while retaining most of the important information (variance).
- Used for high-dimensional numerical data.

Example:

100 features → PCA → 10 principal components.

Use case: Image recognition, data visualization, dimensionality reduction.

2. Linear Discriminant Analysis (LDA)

- Reduces dimensions while maximizing the separation between classes.
- Used for **supervised** classification problems.

Use case: Face recognition, medical diagnosis.

3. Text Vectorization

Converts text into numerical features.

Techniques:

- **Bag of Words (BoW)** – Counts the occurrence of each word.
- **TF-IDF (Term Frequency–Inverse Document Frequency)** – Gives more importance to unique words.
- **Word Embeddings** – Represents words as dense vectors (e.g., Word2Vec, GloVe, FastText).

Example:

"I love AI"

↓

[0.2, 0.8, 0.5, ...]

4. Image Feature Extraction

Extracts meaningful information from images.

Techniques:

- Edge detection
- Color histograms
- Texture features
- CNN (Convolutional Neural Networks)

Use case: Face detection, object recognition.

5. Audio Feature Extraction

Extracts useful information from audio signals.

Examples:

- MFCC (Mel-Frequency Cepstral Coefficients)
- Spectrogram
- Chroma features

Use case: Speech recognition, music classification.

6. Autoencoders

- A type of neural network that learns a compressed representation of data.
- Commonly used for dimensionality reduction and anomaly detection.

7. Independent Component Analysis (ICA)

- Separates mixed signals into independent components.

Use case: Noise removal, signal processing.

Binning:

Binning (also called **discretization**) is a feature engineering technique that **groups continuous numerical values into a small number of intervals (bins)**. Instead of using exact values, we use ranges.

Example

Suppose you have an **Age** column:

Age

18

22

35

47

60

After binning:

Age	Age Group
-----	-----------

18	Young
----	-------

22	Young
----	-------

35	Adult
----	-------

47	Middle-aged
----	-------------

60	Senior
----	--------

Feature Selection

Feature Selection is the process of **selecting the most relevant features (columns)** from a dataset and removing the irrelevant or redundant ones. Unlike **feature extraction**, it **does not create new features**. It only chooses the best existing features.

Example

Suppose you have these features:

Age	Name	Salary	Experience	Employee ID	Target
-----	------	--------	------------	-------------	--------

For predicting salary, you might keep:

- ☒ Age
- ☒ Experience

And remove:

- ☒ Name
- ☒ Employee ID

because they may not help the model.

► Wrapper Method (Feature Selection)

A **Wrapper Method** is a feature selection technique that **selects the best subset of features by repeatedly training and evaluating a machine learning model**. Instead of using only statistical tests (like filter methods), it **"wraps" around the machine learning algorithm** to find the best combination of features.

How it Works

1. Select a subset of features.
2. Train the model using those features.
3. Evaluate the model's performance.
4. Add or remove features.
5. Repeat until the best feature subset is found.

Types of Wrapper Methods:

1. Forward Selection

- Starts with **no features**.
- Adds one feature at a time.
- Keeps the feature that improves model performance the most.
- Stops when adding more features no longer improves the model.

Example:

Features: Age, Salary, Experience, City

Start → {}

↓
Add Age
↓
{Age}
↓
Add Experience
↓
{Age, Experience}
↓
Stop

2. Backward Elimination

- Starts with **all features**.
- Removes one feature at a time.
- Removes the least useful feature.
- Stops when only important features remain.

Example:

{Age, Salary, Experience, City}
↓
Remove City
↓
{Age, Salary, Experience}
↓
Remove Salary
↓
{Age, Experience}

3. Recursive Feature Elimination (RFE)

- Starts with all features.
- Trains the model.
- Ranks features by importance.
- Removes the least important feature.
- Repeats until the desired number of features remains.

Advantages

- Usually provides **high accuracy** because it evaluates actual model performance.
- Finds the best feature combination for a specific model.

Disadvantages

- Computationally expensive because the model is trained many times.
- Slow for datasets with many features.

Iris Dataset

The **Iris dataset** is one of the most famous datasets in machine learning. It was introduced by the British statistician Ronald A. Fisher in **1936** and is commonly used for **classification** tasks.

The Iris dataset is a **benchmark machine learning dataset containing 150 iris flower samples with four numerical features and three species. It is widely used for learning classification algorithms and data visualization techniques.**